

03

PROGRAMMIERUNG VON WEB-ANWENDUNGEN:

HTML & CSS GRUNDLAGEN

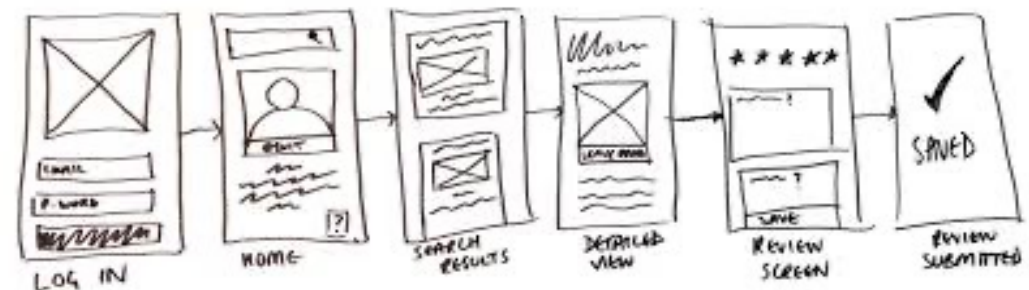
FORTGESCHRITTENES HTML & CSS

JAVASCRIPT UND DYNAMISCHE WEBSEITEN

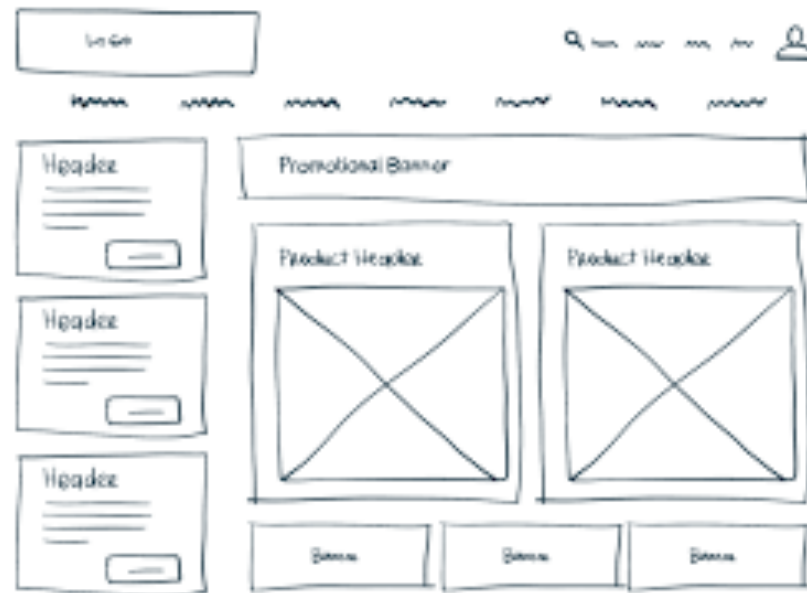
WIREFRAMING – DER BAUPLAN DEINER WEBSEITE

- **Inhalt:**

- Ein Wireframe ist eine einfache Skizze oder ein grober Entwurf deiner Webseite.
- Er zeigt die Anordnung der Elemente, aber nicht das endgültige Design (Farben, Schriftarten, etc.).
- Denke an einen Architekten, der einen Bauplan erstellt, bevor er ein Haus baut.
- Wireframes helfen dir, die Struktur und den Inhalt deiner Webseite zu planen, bevor du mit dem Coden beginnst.
- Du kannst Wireframes von Hand zeichnen oder digitale Tools verwenden.



WIREFRAMING



SCHRITTE ZUR ERSTELLUNG EINES WIREFRAMES

- 1. Seiten auflisten:** Welche Seiten wird deine Webapp haben (Startseite,..., etc.)?
- 2. Inhaltsbereiche identifizieren:** Welche Bereiche (Header, Navigation, Hauptinhalt, Footer) gehören auf jede Seite?
- 3. Elemente skizzieren:** Welche Elemente brauchst du in jedem Bereich (Textfelder, Bilder, Buttons, etc.)?
- 4. Elemente standardisieren:** Verwende einheitliche Größen und Stile für wiederkehrende Elemente.
- 5. Inhalte anordnen:** Ordne die Inhaltsbereiche logisch an (z.B. Navigation oben, Hauptinhalt in der Mitte, Footer unten).
- 6. Feedback einholen:** Zeige dein Wireframe anderen und frage nach Feedback.
- 7. Überarbeiten:** Nimm notwendige Änderungen basierend auf dem Feedback vor.

TIPPS FÜR DIE ERSTELLUNG EINES WIREFRAMES

- **Werkzeuge:** Verwende Stift und Papier oder ein Wireframe-Tool (z.B. Balsamiq, Figma, Adobe XD).
- **Einfachheit:** Halte das Wireframe einfach und leicht verständlich.
- **Fokus auf Struktur:** Konzentriere dich auf die Struktur und Funktionalität, nicht auf das Aussehen.
- **Feedback:** Hole dir Feedback von anderen ein, um verschiedene Perspektiven zu erhalten.
- **Iteration:** Überarbeite dein Wireframe basierend auf dem Feedback und neuen Erkenntnissen.

HTML – DAS GRUNDGERÜST DEINER WEBSEITE

- HTML (Hypertext Markup Language) ist die Standardsprache zur Erstellung von Webseiten.
- HTML definiert die Struktur einer Webseite, ähnlich wie das Skelett eines Körpers.
- HTML-Elemente sind die Bausteine deiner Seite: Überschriften, Absätze, Bilder, Links...
- HTML-Tags geben diesen Elementen Bedeutung.



HTML

```
<h1>Willkommen auf meiner Seite!</h1>  
<p>Dies ist ein Absatz.</p>  

```

HTML-ELEMENTE UND -TAGS

- Wichtige Elemente:
 - `<html>` : Das Wurzelement, umschließt alles andere.
 - `<head>` : Enthält Meta-Informationen (nicht sichtbar).
 - `<title>` : Der Titel deiner Seite (erscheint im Browser-Tab).
 - `<body>` : Der sichtbare Inhalt deiner Seite.
 - `<h1>` bis `<h6>` : Überschriften (unterschiedliche Ebenen).
 - `<p>` : Absätze.
 - `<a>` : Links zu anderen Seiten (`href` für die Adresse).
 - `<img>` : Bilder (`src` für den Bildpfad, `alt` für Beschreibung).
 - `<div>` : Ein Container, um Elemente zu gruppieren.
 - `<span>` : Ein Inline-Container, um Teile von Text zu formatieren.
 - `<ul>` , `<ol>` , `<li>` : Listen (ungeordnet, geordnet, Einträge).
- **Nicht vergessen:** `<meta charset="UTF-8">` im `<head>` für korrekte Zeichenanzeige!

AUFBAU EINER EINFACHEN WEBSEITENSTRUKTUR

— Jede Webseite hat eine Grundstruktur

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>Meine Filmseite</title>
    <meta charset="UTF-8">
  </head>
  <body>
    </body>
</html>
```

PROJEKT (TEIL 1): FILMLISTEN-SEITE

- **Aufgabe:** Erstelle das HTML-Grundgerüst für deine Filmlisten-Seite.
- **Was brauchst du?**
 - Eine Überschrift (`<h1>`) für den Seitentitel.
 - Eine Liste (`<ul>` oder `<ol>`) mit Platzhaltern für die Filme.
 - Denke an die richtige Struktur: `<html>` , `<head>` , `<body>` .

GRUNDLEGENDES STYLING MIT CSS

- **Grundlegende CSS-Eigenschaften:**

- `color` : Textfarbe (z.B. `color: red;` , `color: #ff0000;`)
- `background-color` : Hintergrundfarbe (z.B. `background-color: yellow;`)
- `font-family` : Schriftart (z.B. `font-family: Arial, sans-serif;`)
- `font-size` : Schriftgröße (z.B. `font-size: 16px;`)
- `width` , `height` : Breite und Höhe (z.B. `width: 50%;` , `height: 200px;`)
- `margin` , `padding` : Außen- und Innenabstände (z.B. `margin: 10px;` ,
`padding: 20px;`)
- `border` : Rahmen (z.B. `border: 1px solid black;`)
- `text-align` : Textausrichtung (z.B. `text-align: center;`)

CSS

```
body {  
    font-family: Arial, sans-serif;  
    background-color: #f0f0f0;  
}  
  
h1 {  
    color: #333;  
    font-size: 2em;  
    text-align: center;  
}
```

PROJEKT (TEIL 2): STYLING DER FILMLISTEN-SEITE (CSS)

- **Inhalt:**

- **Aufgabe:** Gestalte deine Filmlisten-Seite mit CSS!
- **Was kannst du tun?**
 - Wähle passende Farben für Hintergrund, Text und Links.
 - Verwende verschiedene Schriftarten für Überschriften und Text.
 - Gestalte die Filmeinträge (Größe, Rahmen, Abstände).
 - Arrangiere die Filmeinträge (Liste, Grid, etc.).
 - Denke an Responsive Design, damit die Seite auf verschiedenen Geräten gut aussieht.

PROGRAMMIERUNG VON WEB-ANWENDUNGEN:

FORTGESCHRITTENES HTML & CSS

FORMULARE, LAYOUTS UND RESPONSIVE DESIGN

HTML-FORMULARE – BENUTZEREINGABEN SAMMELN

- Formulare ermöglichen es Benutzern, Daten einzugeben und an den Server zu senden.
- Wichtige Elemente:
 - `<form>` : Definiert das Formular.
 - `<input>` : Erstellt verschiedene Arten von Eingabefeldern (Text, Passwort, Checkbox, Radiobutton, etc.).
 - `<textarea>` : Erstellt ein mehrzeiliges Textfeld.
 - `<select>` : Erstellt ein Dropdown-Menü.
 - `<button>` : Erstellt einen Button zum Absenden des Formulars.

HTML

```
<form action="/submit" method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name"><br><br>
  <label for="email">E-Mail:</label>
  <input type="email" id="email" name="email"><br><br>
  <input type="submit" value="Absenden">
</form>
```

CSS FLEXBOX – FLEXIBLE LAYOUTS LEICHT GEMACHT

- Flexbox ist ein CSS-Layout-Modul, das das Erstellen flexibler und responsiver Layouts erleichtert.
- Flex-Container und Flex-Items:
 - Der Flex-Container ist das übergeordnete Element, das die Flex-Items enthält.
 - Flex-Items sind die Kindelemente des Flex-Containers.

CSS

```
.container {  
  display: flex;  
  flex-direction: row;  
  justify-content: space-around;  
  align-items: center;  
}
```

CSS FLEXBOX – FLEXIBLE LAYOUTS LEICHT GEMACHT

◦ Wichtige Eigenschaften:

- `display: flex` : Macht ein Element zum Flex-Container.
- `flex-direction` : Legt die Richtung fest, in der die Flex-Items angeordnet werden (row, column, row-reverse, column-reverse).
- `justify-content` : Richtet die Flex-Items entlang der Hauptachse aus (flex-start, flex-end, center, space-between, space-around).
- `align-items` : Richtet die Flex-Items entlang der Querachse aus (flex-start, flex-end, center, baseline, stretch).
- `flex-wrap` : Legt fest, ob die Flex-Items umgebrochen werden sollen, wenn sie nicht in eine Zeile passen (nowrap, wrap, wrap-reverse).
- `flex-grow` , `flex-shrink` , `flex-basis` : Steuern, wie Flex-Items den verfügbaren Platz im Container nutzen.

CSS

```
.container {  
  display: flex;  
  flex-direction: row;  
  justify-content: space-around;  
  align-items: center;  
}
```

CSS GRID - KOMPLEXE LAYOUTS IM GRIFF

- CSS Grid ist ein weiteres Layout-Modul, das sich besonders für komplexere Layouts eignet.
- Grid-Container und Grid-Items:
 - Der Grid-Container ist das übergeordnete Element, das die Grid-Items enthält.
 - Grid-Items sind die Kindelemente des Grid-Containers.
- Wichtige Eigenschaften:
 - `display: grid` : Macht ein Element zum Grid-Container.
 - `grid-template-columns` , `grid-template-rows` : Definieren die Spalten und Zeilen des Grids.
 - `grid-gap` : Legt den Abstand zwischen den Grid-Items fest.
 - `justify-items` , `align-items` : Richten die Grid-Items innerhalb ihrer Zellen aus.
 - `grid-area` : Platziert ein Grid-Item in einem bestimmten Bereich des Grids.

CSS GRID - KOMPLEXE LAYOUTS IM GRIFF

◦ Wichtige Eigenschaften:

- `display: grid` : Macht ein Element zum Grid-Container.
- `grid-template-columns` , `grid-template-rows` : Definieren die Spalten und Zeilen des Grids.
- `grid-gap` : Legt den Abstand zwischen den Grid-Items fest.
- `justify-items` , `align-items` : Richten die Grid-Items innerhalb ihrer Zellen aus.
- `grid-area` : Platziert ein Grid-Item in einem bestimmten Bereich des Grids.

```
container{
  display: grid;
  grid-template-columns: 1fr 2fr 1fr; /* Drei
  Spalten, die mittlere doppelt so breit */
  grid-gap: 20px;
}

.item1{
  grid-area: 1 / 1 / 2 / 2; /* Startzeile / Startspalte /
  Endzeile / Endspalte */
}

.item2{
  grid-area: 1 / 2 / 3 / 3;
}
```

RESPONSIVE DESIGN – WEBSEITEN FÜR ALLE GERÄTE

- CSS Grid ist ein weiteres Layout-Modul, das sich besonders für komplexere Layouts eignet.
- Grid-Container und Grid-Items:
 - Der Grid-Container ist das übergeordnete Element, das die Grid-Items enthält.
 - Grid-Items sind die Kindelemente des Grid-Containers.
- Wichtige Eigenschaften:
 - `display: grid` : Macht ein Element zum Grid-Container.
 - `grid-template-columns` , `grid-template-rows` : Definieren die Spalten und Zeilen des Grids.
 - `grid-gap` : Legt den Abstand zwischen den Grid-Items fest.
 - `justify-items` , `align-items` : Richten die Grid-Items innerhalb ihrer Zellen aus.
 - `grid-area` : Platziert ein Grid-Item in einem bestimmten Bereich des Grids.

PROJEKT (TEIL 3): ERSTELLUNG DES LAYOUTS FÜR DIE FILMDETAIL-SEITE (HTML, CSS).

- **Aufgabe:** Erstelle das Layout für eine einzelne Filmseite mit HTML und CSS.
- **Was brauchst du?**
 - Bereich für das Filmplakat (`<img>`)
 - Überschrift mit dem Filmtitel (`<h1>`)
 - Absätze oder andere Elemente für Beschreibung, Genre, Schauspieler, etc. (`<p>` , `<span>` , vielleicht sogar eine Liste)
 - Ein Bereich für die Bewertung (später interaktiv mit JavaScript/React)
 - Eventuell Platz für Kommentare/Reviews (falls ihr diese Funktion später einbaut)
- **Denk daran:**
 - Verwende semantische HTML-Elemente, um die Bedeutung der Inhalte zu verdeutlichen (z.B. `<article>` für den Film, `<figure>` für das Poster).
 - Überlege dir ein ansprechendes Layout mit CSS.
 - Teste dein Layout auf verschiedenen Bildschirmgrößen (Responsive Design!).

PROGRAMMIERUNG VON WEB-ANWENDUNGEN:

JAVASCRIPT UND DYNAMISCHE WEBSEITEN

INTERAKTIVITÄT UND DATENVERARBEITUNG

JAVASCRIPT BASICS

- **Variablen:** Zum Speichern von Daten (z.B. `let name = 'Max';`)
- **Datentypen:** Zahlen, Strings, Booleans, Arrays, Objekte, etc.
- **Operatoren:** Zum Durchführen von Berechnungen und Vergleichen (z.B. `+`, `-`, `*`, `/`, `>`, `<`, `==`, `&&`, `||`)
- **Funktionen:** Wiederverwendbare Codeblöcke (z.B. `function sayHello(name) { console.log('Hallo, ' + name + '!'); }`)
- **Kontrollfluss:** `if / else`, `switch`, `for`, `while` -Schleifen

JavaScript

```
// Variablen und Datentypen
let name = "Max";
let age = 30;
let isStudent = true;

// Funktion
function greet(name) {
    console.log("Hallo, " + name + "!");
}

// Kontrollfluss (if/else)
if (isStudent) {
    console.log("Du bekommst einen Studentenrabatt!");
} else {
    console.log("Der normale Preis gilt.");
}
```

DOM MANIPULATION

- **Das DOM (Document Object Model):** Eine baumartige Darstellung des HTML-Dokuments.
- **Elemente auswählen:** `document.getElementById()`, `document.querySelector()`, etc.
- **Inhalte ändern:** `element.textContent`, `element.innerHTML`, etc.
- **Stile ändern:** `element.style.color`, `element.classList.add()`, etc.
- **Events behandeln:** `element.addEventListener('click', function() { ... });`

JavaScript

```
// Element auswählen
const myHeading = document.getElementById("myHeading");

// Inhalt ändern
myHeading.textContent = "Neuer Titel!";

// Stile ändern
myHeading.style.color = "blue";

// Event Listener hinzufügen
const myButton = document.getElementById("myButton");
myButton.addEventListener("click", function() {
    alert("Button wurde geklickt!");
});
```

ASYNCHRONES JAVASCRIPT

- JavaScript ist **single-threaded**, d.h. es kann nur eine Aufgabe gleichzeitig ausführen.
- **Asynchrone Operationen** (z.B. Laden von Daten, Netzwerkaufrufe) blockieren den Hauptthread nicht.
- **Callbacks**: Ältere Methode zur Behandlung asynchroner Ereignisse, kann aber zu "Callback Hell" führen.
- **Promises**: Ein modernerer Ansatz, der eine klarere Struktur bietet und Fehlerbehandlung erleichtert.
- **Async/await**: Eine noch lesbarere Syntax, die auf Promises basiert und asynchronen Code wie synchronen Code aussehen lässt.

• Codebeispiel (Promise):

```
fetch('https://api.example.com/data')  
.then(response => response.json())  
.then(data => console.log(data))  
.catch(error => console.error('Fehler:', error));
```

FETCHING DATA WITH FETCH API

- Die Fetch API ist eine moderne Methode, um Daten von einem Server oder einer API abzurufen.
- Ermöglicht das Senden von HTTP-Anfragen (GET, POST, PUT, DELETE) und das Empfangen von Antworten.
- Gibt ein Promise zurück, das entweder die Antwortdaten oder einen Fehler enthält.

JavaScript

```
async function fetchData() {  
  try {  
    const response = await fetch('https://api.example.com/data')  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error('Fehler  
beim Laden der Daten:', error);  
  }  
}
```

PROJEKT (TEIL 4): INTERAKTIVITÄT ZUR FILMLISTEN-SEITE

- **Aufgabe:** Füge deiner Filmlisten-Seite Interaktivität hinzu.
- **Mögliche Funktionen:**
 - Beim Klick auf einen Filmtitel werden weitere Details angezeigt (Beschreibung, Genre, etc.).
 - Eine Suchfunktion, um Filme nach Titel zu filtern.
 - Ein Button, um die Liste nach Bewertung oder Erscheinungsjahr zu sortieren.
- **Tipps:**
 - Verwende JavaScript, um auf Klicks und andere Events zu reagieren.
 - Ändere den Inhalt oder die Stile von Elementen im DOM, um die Interaktivität zu implementieren.
 - Denke daran, dass du möglicherweise Daten von der API neu laden oder filtern musst, um die Suchergebnisse oder Sortierung anzuzeigen.

PROJEKT (TEIL 5): LADEN VON FILMDATEN UND ANZEIGEISTEN-SEITE

- **Aufgabe:** Verwende die Fetch API, um Filmdaten von einer externen API abzurufen und in deiner Filmlisten-Seite anzuzeigen.
- **Schritte:**
 1. Wähle eine geeignete Film-API aus (z.B. The Movie Database (TMDb)).
 2. Verwende `fetch`, um eine Anfrage an die API zu senden.
 3. Verarbeite die Antwort und extrahiere die benötigten Filmdaten (Titel, Poster, etc.).
 4. Aktualisiere deine React-Komponenten, um die Filmdaten anzuzeigen.
- **Tipps:**
 - Verwende `async/await` für eine bessere Lesbarkeit.
 - Handle Fehler, falls die API nicht erreichbar ist oder ungültige Daten zurückgibt.
 - Verwende Platzhalter oder Ladeanimationen, während die Daten geladen werden.